

π Tantum

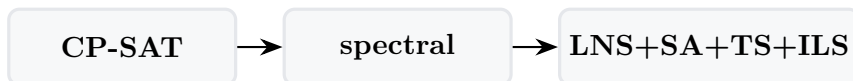
alias *Tempus Tantum*

sistema di generazione orario scolastico

Appendici tecniche

Parte II — per sviluppatori

Riferimento tecnico completo: architettura del sistema, modello dati con relazioni e vincoli FK, API REST, grammatica formale del DSL e traduzione $\text{AST} \rightarrow \text{CP-SAT}$, strategie di ottimizzazione (CP-SAT, decomposizione spettrale, metaeuristiche), diagnostica statistica, guida all'estensione del sistema.



Giovanni Borghi

3 maggio 2026

Indice

1	Architettura	3
1.1	Stack	3
1.2	Diagramma a blocchi	3
1.3	Struttura cartelle	3
1.4	Lifecycle del backend	4
2	Modello dati	5
2.1	Famiglie di tabelle	5
2.2	Diagramma relazioni	5
2.3	Migrazioni idempotenti	5
2.4	Pickle dell'engine	6
3	DSL generico per i vincoli	7
3.1	Filosofia	7
3.2	Grammatica BNF	7
3.3	Sorgenti iterabili	8
3.4	Galleria di esempi	8
3.4.1	Vincoli docente (10 esempi)	8
3.4.2	Vincoli classe (5 esempi)	9
3.4.3	Vincoli aula (5 esempi)	10
3.4.4	Vincoli materia (3 esempi)	11
3.4.5	Vincoli gruppo / studenti (3 esempi)	11
3.4.6	Vincoli relazionali / coupling (4 esempi)	11
3.5	Errori comuni	12
4	Tecniche di ottimizzazione avanzate	13
4.1	Adaptive Large Neighborhood Search (ALNS)	13
4.2	Variable Neighborhood Search (VNS)	13
4.3	Hall's theorem pre-check	13
4.4	Column Generation (Dantzig-Wolfe)	13
4.5	Lagrangian Relaxation	13
5	Diagnostica statistica	14
5.1	Sensitivity Monte Carlo	14
5.2	Analisi bipartito	14
5.3	Correlazioni e regressioni	14
5.4	Distribuzioni	14
5.5	Run telemetry	14

6	API REST	15
6.1	Pattern CRUD	15
6.2	Esempi curl	15
6.3	Riferimento gruppi di endpoint	16
7	Estendere il sistema	17
7.1	Aggiungere una nuova tabella + CRUD	17
7.2	Migrazione idempotente	17
7.3	Aggiungere un nuovo tipo di vincolo	17
7.4	Aggiungere un nuovo step di ottimizzazione	17
7.5	Testing	18
A	Repository GitHub	19
B	Glossario discorsivo dei metodi	20
B.1	CP-SAT (Constraint Programming over SAT)	20
B.2	Decomposizione spettrale	20
B.3	LNS (Large Neighborhood Search)	21
B.4	ALNS (Adaptive LNS)	21
B.5	SA (Simulated Annealing)	22
B.6	TS (Tabu Search)	22
B.7	ILS (Iterated Local Search)	22
B.8	VNS (Variable Neighborhood Search)	22
B.9	Hall's theorem pre-check	23
B.10	Lagrangian Relaxation con sub-gradient	23
B.11	Column Generation (Dantzig-Wolfe)	24
B.12	Monte Carlo Sensitivity	24
B.13	Modularità, betweenness, densità (analisi del bipartito)	24
B.14	Distribuzioni e istogrammi	25
B.15	Correlazioni e regressioni	25
B.16	DSL generico	25
B.17	Stati 5-colori delle matrici	26
B.18	Tag aule e tag studenti	26
B.19	Run telemetry e visualizzazione live	27
B.20	Lockati: lezioni che non si toccano	27
C	Reference	28

1. Architettura

1.1 Stack

- **Backend:** Python 3.11+, FastAPI, SQLAlchemy 2.0, Pydantic 2, uvicorn, OR-Tools, NumPy, scikit-learn, Faker.
- **Frontend:** SvelteKit (Svelte 4), Vite 5, Tailwind CSS, JavaScript puro.
- **DB:** SQLite via SQLAlchemy. Migrazioni idempotenti nel codice (no Alembic).
- **Engine I/O:** pickle Python; `engine_io.py` converte fra DB e dict per il solver.

1.2 Diagramma a blocchi

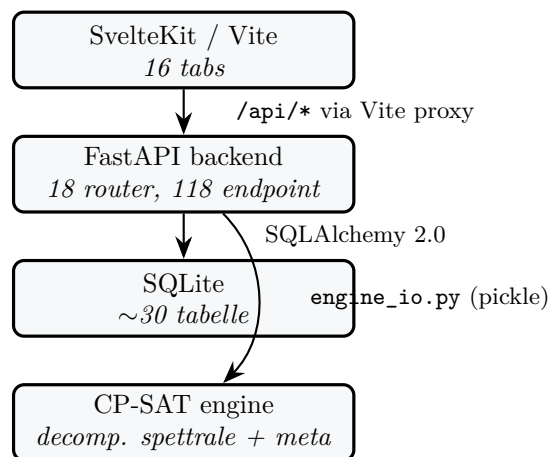


Figura 1.1: Architettura a blocchi della webapp.

1.3 Struttura cartelle

```
timetable/  
+-- README.md -- landing page  
+-- docs/ -- manuale tecnico  
+-- experiments/ -- solver code + pickle snapshots  
| +-- cpsat_v2_assignment.py -- Phase A  
| +-- cpsat_v2_timetable.py -- Phase B  
| +-- decomposition_spectral_v2.py  
| +-- metaheuristics.py -- LNS / SA / TS / ILS  
| +-- classroom_assignment.py -- Phase 4  
+-- schedule/ -- legacy single-file prototypes  
+-- webui/  
| +-- start.bat -- launcher Windows
```

```
| +-- start.sh -- launcher Linux/macOS
| +-- stop.sh -- stop helper
| +-- backend/ -- FastAPI app
| +-- frontend/ -- SvelteKit
| +-- data/timetable.db -- SQLite
| +-- docs/ -- guide brevi user-facing
+-- proposals/ -- design notes + benchmarks
+-- *.pdf -- reference papers
```

1.4 Lifecycle del backend

webui/backend/main.py definisce un lifespan che chiama `init_db()` allo startup. Esso fa:

1. `Base.metadata.create_all(bind=engine)` – crea le tabelle mancanti.
2. `_apply_lightweight_migrations()` – ALTER TABLE idempotenti per i campi aggiunti in versioni successive (esempio: `school_classes.curriculum_id`, `teachers.last_name/first_name/nickname` ecc.). Ogni ALTER è guardato da `PRAGMA table_info` per non duplicare.

2. Modello dati

Tutto vive in `webui/backend/models.py`. Le tabelle si raggruppano in cinque famiglie tematiche.

2.1 Famiglie di tabelle

Anagrafiche

`subjects`, `teachers`, `school_classes`, `classrooms`, `curricula`, `students`, `study_groups`.

Tag (M-N)

`classroom_tags` + `classroom_tag_assignments`, `student_tags` + `student_tag_assignments`.
Nome unique lower-cased, descrizione opzionale, cancellazione cascata.

Assegnazione

`class_subjects`, `curriculum_subject_hours`, `teacher_subjects`, `subject_group_weights`,
`assignments`.

Disponibilità / vincoli

`teacher_unavailability`, `class_unavailability`, `classroom_unavailability`, `logical_unavailabilitie`,
`curriculum_logical_constraints`, `classroom_subject_preferences`, `teacher_classroom_preferences`,
`coteaching_rules`.

Soluzioni e scheduling

`solutions`, `lessons`, `day_counts`.

Coverage e Run

`absences`, `substitute_assignments`, `runs`, `run_logs`, `app_state`.

Viste salvate

`saved_views`: query DSL + sort multi-livello salvati per una entità lista, esposti dal dropdown
"Viste salvate" del componente *SortableQueryableList*. UNIQUE (`entity`, `name`).

2.2 Diagramma relazioni

2.3 Migrazioni idempotenti

Pattern (in `db.py`):

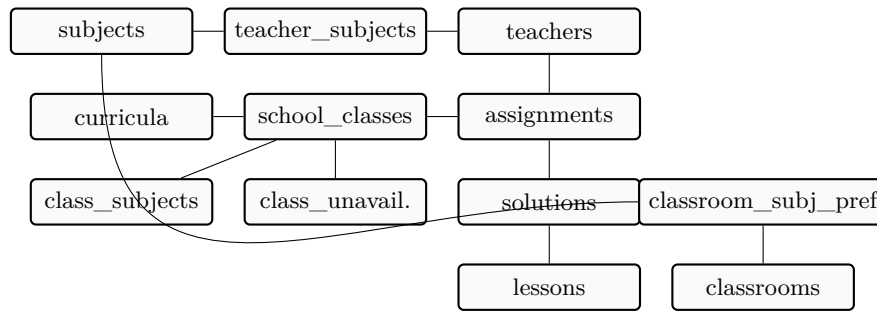


Figura 2.1: Relazioni rilevanti del modello dati (semplificato).

```

if insp.has_table("school_classes") \
    and not has_column("school_classes", "curriculum_id"):
    conn.execute(text(
        "ALTER TABLE school_classes ADD COLUMN curriculum_id INTEGER"
    ))

```

Ogni ALTER è guardato da una check su PRAGMA table_info. Quando aggiungi una colonna nuova, segui lo stesso pattern: il DB preesistente sopravvive senza interventi manuali.

2.4 Pickle dell'engine

engine_io.py converte fra DB e tre forme pickle:

```

school = {
    'profile': str,
    'classes': [{name, year, section, curriculum, subjects:{subj:ore}}],
    'teachers': [{name, group, max_hours, free_day,
                  weights:{subj:int}}],
    'cconcorsopersubject': {subj: {group: weight}},
    'curriculum_scores': {curriculum: int},
    'curricula': [...],
    'students': [...],
    'groups': [...],
}
profs = {
    teacher_name: {
        'classi': {class_name: {subject: {'ore': N}}},
        'glibero': [d1, d2, d3]
    }
}
solution = {(prof, class, subj, day, hour): 0|1}

```

3. DSL generico per i vincoli

Linguaggio compatto, dichiarativo, totalmente componibile per esprimere vincoli HARD/SOFT/PREFERITO/ENFORCED su qualunque combinazione logica di predicati sul modello dei dati. Implementato in `webui/backend/utils/general_dsl.py` (parser ricorsivo discendente + AST + evaluator post-hoc, ~250 LOC, no dipendenze esterne). Esposto via `POST /api/constraints/general`, validato live da `POST /api/constraints/general/validate`, accessibile dal modal “+ Nuovo vincolo DSL” del tab Vincoli.

3.1 Filosofia

Prima del DSL generico il sistema aveva quattro sub-DSL specializzati (query, logical, objective, introspettivo). Stessa grammatica implementata 4 volte. Il DSL generico unifica tutto sotto una sola grammatica con quattro “sapori” di compilazione (LIST, LOGIC, OBJECTIVE, EVAL): *un parser, molti compilatori*. La sintassi di `forall x in S where Filter: Body` e dei predicati atomici (`==`, `!=`, `<`, `in`, ...) è identica in tutti i contesti; cambia solo il backend di traduzione.

Il DSL è interpretato *post-hoc*: parsea l’espressione e la valuta DOPO che la pipeline (Phase A, Phase B, metaeuristiche) ha prodotto una soluzione candidata. Le violazioni HARD sono rilevate dal Feasibility Check; le SOFT alimentano lo score globale.

3.2 Grammatica BNF

```
expr ::= iff_expr
iff_expr ::= implies_expr ( "<=>" implies_expr ) *
implies_expr ::= or_expr ( "=>" or_expr ) *
or_expr ::= and_expr ( ("OR"|"or") and_expr ) *
and_expr ::= not_expr ( ("AND"|"and") not_expr ) *
not_expr ::= ("NOT"|"not") not_expr | atom

atom ::= quant_expr | count_expr | comparison
      | call | bool_lit | "(" expr ")"

quant_expr ::= ("forall"|"exists") IDENT "in" source
            ["where" or_expr] ":" expr

count_expr ::= "count" IDENT "in" source ["where" or_expr]
            ( ":" comparison | op value )

source ::= IDENT -- literal name
        | IDENT ( "." IDENT ) + -- path (runtime)

comparison ::= value ( op value
                    | "in" "[" value ( "," value ) * "]"
                    | "in" path
                    | "not_in" "[" value ( "," value ) * "]"
```



```

        | "not_in" path )

op ::= "==" | "!=" | "<" | "<=" | ">" | ">="
value ::= IDENT ( "." IDENT )* | NUM | STRING
        | BOOL | function_call
function_call ::= IDENT "(" [arg_list] ")"
arg_list ::= arg ("," arg)*
arg ::= IDENT "==" value | value

```

Precedenze (dal più basso al più alto): <=>, ==>, OR, AND, NOT, confronti, . (member access). Le parentesi sovrascrivono.

3.3 Sorgenti iterabili

Sorgente	Tipo elemento
lessons	lezioni della soluzione attiva
assignments	cattedre (Assignment rows)
teachers	docenti (name, group, max_hours, subject)
classes	classi (name, year, curriculum, n_students)
classrooms	aule (name, kind, type, tags)
students	studenti (con tags[] e groups[])
subjects	materie
curricula	indirizzi di studio
groups	gruppi articolati
slots	le 36 (day, hour) della settimana
days	i 6 giorni (Lun..Sab; index 1..6)
hours	le 6 ore (8..13)

Sorgente-path: dalla v0.5 puoi scrivere `exists g in s.groups: g.name == "X"` dove `s.groups` e' un attributo che restituisce una lista; il parser accetta qualunque path puntato dopo `in`.

3.4 Galleria di esempi

Ogni esempio sotto e' stato validato live contro il parser corrente; copia-incolla nell'editor del modal "+ Nuovo vincolo DSL" e funziona.

3.4.1 Vincoli docente (10 esempi)

1. Borghi non insegna mai in 1A alla 6a ora:

```

forall l in lessons where l.teacher == Borghi
                        and l.class == 1A:
    l.hour != 13

```

2. Borghi: max 4 ore/giorno totali:

```

forall d in days:
    count l in lessons where l.teacher == Borghi
                        and l.day == d.index: l <= 4

```

3. Lab fisica: ogni docente di Fisica esattamente 1 ora a settimana in lab_fisica:

```

forall t in teachers where t.subject == "Fisica":
    count l in lessons where l.teacher == t
                        and l.classroom.type == "lab_fisica": l == 1

```

4. Tetto cattedra Inglese: max 18h/settimana:

```
forall t in teachers where t.subject == "Inglese":
    count l in lessons where l.teacher == t: l <= 18
```

5. Rossi non lavora il sabato (HARD):

```
forall l in lessons where l.teacher == Rossi
    and l.day == 6:
    false
```

6. Rossi: al più 1 sesta ora/settimana:

```
count l in lessons where l.teacher == Rossi
    and l.hour == 13: l <= 1
```

7. Tutti A050: max 5 ore/giorno:

```
forall t in teachers where t.group == "A050":
    forall d in days:
        count l in lessons where l.teacher == t
            and l.day == d.index: l <= 5
```

8. Coupling Rossi 3A $\Rightarrow$ Bianchi 3B:

```
(exists l in lessons:
    l.teacher == Rossi and l.class == 3A)
=> (exists l in lessons:
    l.teacher == Bianchi and l.class == 3B)
```

9. Rossi mai in palestra:

```
forall l in lessons where l.teacher == Rossi:
    l.classroom.type != "palestra"
```

10. Tutti i prof di mate: almeno una lezione di mattina:

```
forall t in teachers where t.subject == "Matematica":
    exists l in lessons where l.teacher == t:
        l.hour <= 9
```

3.4.2 Vincoli classe (5 esempi)

1. La 1A non lavora il sabato:

```
forall l in lessons where l.class == 1A: l.day != 6
```

2. Le prime: max 5 ore/giorno:

```
forall c in classes where c.year == 1:
    forall d in days:
        count l in lessons where l.class == c.name
            and l.day == d.index: l <= 5
```

3. Quarte/quinte: due ore CONSECUTIVE di mate:

```
forall c in classes where c.year >= 4:
  exists s1 in slots: exists s2 in slots:
    consecutive(s1, s2)
    and lesson(class==c.name, day==s1.day,
               hour==s1.hour, subject==Matematica)
    and lesson(class==c.name, day==s2.day,
               hour==s2.hour, subject==Matematica)
```

4. Linguistico anno 1: niente sabato:

```
forall l in lessons
  where l.class.curriculum == Linguistico
  and l.day == 6: false
```

5. 1A: religione solo nelle prime 4 ore:

```
forall l in lessons where l.class == 1A
  and l.subject == Religione:
  l.hour <= 11
```

3.4.3 Vincoli aula (5 esempi)

1. Lab Fisica 1: una sola lezione per slot:

```
forall s in slots:
  count l in lessons
    where l.classroom == "Lab Fisica 1"
    and l.day == s.day and l.hour == s.hour:
  l <= 1
```

2. Matematica solo in aule taggate “matematica”:

```
forall l in lessons where l.subject == Matematica:
  "matematica" in l.classroom.tags
```

3. Storia delle terze: aula con proiettore:

```
forall l in lessons where l.subject == Storia
  and l.class.year >= 3:
  "proiettore" in l.classroom.tags
```

4. La palestra ospita SOLO Scienze motorie:

```
forall l in lessons
  where l.classroom.type == "palestra":
  l.subject == Scienzemotorie
```

5. Lab chimica: max 2 lezioni concorrenti:

```
forall s in slots:
  count l in lessons
    where l.classroom.type == "lab_chimica"
    and l.day == s.day and l.hour == s.hour:
  l <= 2
```

3.4.4 Vincoli materia (3 esempi)

1. Mate distribuita: max 2 ore/giorno per classe:

```
forall c in classes:
  forall d in days:
    count l in lessons
      where l.class == c.name
        and l.subject == Matematica
        and l.day == d.index: l <= 2
```

2. Mate preferibilmente entro la 5a ora:

```
forall l in lessons where l.subject == Matematica:
  l.hour <= 12
```

3. Educazione fisica solo in palestra:

```
forall l in lessons
  where l.subject == Educazione fisica:
    l.classroom.type == "palestra"
```

3.4.5 Vincoli gruppo / studenti (3 esempi)

1. Studenti BES devono essere in un gruppo Sostegno (uso di `exists g in s.groups`):

```
forall s in students where "BES" in s.tags:
  exists g in s.groups: g.name == "Sostegno"
```

2. Studenti con debito mate quarto → gruppo Recupero:

```
forall s in students
  where "debito_matematica_4" in s.tags:
    exists g in s.groups:
      g.name == "Recupero Matematica"
```

3. Studenti BES: nessuna lezione il sabato:

```
forall l in lessons:
  forall s in students
    where l.class == s.class
      and "BES" in s.tags:
        l.day != 6
```

3.4.6 Vincoli relazionali / coupling (4 esempi)

1. Rossi in Aula 12: solo Matematica:

```
forall l in lessons where l.teacher == Rossi
  and l.classroom == "Aula 12":
  l.subject == Matematica
```

2. Se Rossi in 3A allora deve usare lab fisica almeno una volta:

```
(exists l in lessons:
  l.teacher == Rossi and l.class == 3A)
=>
(exists l2 in lessons:
  l2.teacher == Rossi and l2.class == 3A
  and l2.classroom.type == "lab_fisica")
```

3. Inglese alle prime: max 3 ore/sett per (docente,classe):

```
forall t in teachers where t.subject == "Inglese":  
  forall c in classes where c.year == 1:  
    count l in lessons  
      where l.teacher == t  
        and l.class == c.name: l <= 3
```

4. Religione alle prime: mai in 1a ora:

```
forall l in lessons where l.subject == Religione  
  and l.class.year == 1:  
  l.hour != 8
```

3.5 Errori comuni

- **Atteso COLON, trovato OP (=)** – usa == nei confronti dopo **where** (= singolo e' lecito, ma confonde il parser quando appare in contesti ambigui).
- **sorgente sconosciuta 'foo'** – sorgente non in `_VALID_SOURCES` e non e' un path raggiungibile via env. Controlla l'ortografia.
- **oltre 1000000 atomi: probabile esplosione combinatoria** – quantificatori troppo annidati. Sposta filtri nel **where** più esterno per ridurre il set; oppure splitta il vincolo in più vincoli salvati separatamente.

Vedere `docs/general_dsl.md` per la guida completa con sezioni 1–12 (galleria di 30+ esempi commentati, troubleshooting esteso, reference tabellare di tutti gli attributi per ciascuna entità).

4. Tecniche di ottimizzazione avanzate

Oltre alle metaeuristiche classiche (LNS / SA / TS / ILS) il progetto include cinque tecniche aggiuntive. Vedere `docs/optimization_strategies.md` per la specifica completa.

4.1 Adaptive Large Neighborhood Search (ALNS)

`experiments/alns.py`. Sei destroy operator (`random_window`, `day_cluster`, `worst_fit_day`, `teacher_day`, `classroom_day`, `single_class_week`) e tre repair operator (`cp_sat_window`, `greedy_by_soft`, `bfs_fill_back`) selezionati con roulette wheel sopra exponentially-decayed scores; acceptance SA-like.

4.2 Variable Neighborhood Search (VNS)

`experiments/vns.py`. Cicla quattro vicinati di dimensione crescente (1-swap, 2-swap, 3-chain, k-opt con $k \in [4, 6]$). Ad ogni miglioramento riparte dal vicinato 1; termina quando un'intera passata 1→4 non trova nulla.

4.3 Hall's theorem pre-check

`experiments/diagnostics/hall_check.py`. Diagnostico sincrono pre-Phase-A. Tre controlli:

1. per (class, subject) coverage,
2. subject supply vs demand,
3. Hall sampling random-subset con sottoinsiemi esclusivi.

Esposto in *tre* punti UI: bottone inline in PhaseACard, riga in AdvancedTechniquesCard, sezione del tab Statistiche.

4.4 Column Generation (Dantzig-Wolfe)

`experiments/column_generation.py`. Master LP via `scipy.linprog` HiGHS; sotto-problema per docente. Skeleton single-iteration per scuole molto grandi (> 200 classi). Default OFF nella pipeline.

4.5 Lagrangian Relaxation

`experiments/lagrangian.py`. Rilassa i ponti inter-cluster e li dualizza con multipliers λ_b ; aggiornamento via subgradient ascent $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$ con $\alpha_k = \alpha_0 / (1 + k)$. Per-iteration refinement via SA.

5. Diagnostica statistica

Il tab `/diagnostics` aggrega tutte le analisi sul modello e sulla soluzione attiva. Vedere `docs/diagnostics.md`. Visualizzazioni con Apache ECharts (lazy-loaded), tema *pitantum* (palette indaco / oro / avorio / terra-di-siena).

5.1 Sensitivity Monte Carlo

Genera N perturbazioni feasible (sequenze di mosse atomiche random accettate solo se HARD soddisfatto) e calcola statistiche SOFT. Coefficiente di variazione $< 0.05 \Rightarrow$ soluzione vicina all'ottimo locale.

5.2 Analisi bipartito

Tre metriche networkx: densita' del grafo classe $\times$ docente, modularita' Newman-Girvan greedy, top-K betweenness centrality.

5.3 Correlazioni e regressioni

Tre regressioni statsmodels: OLS `load_vs_gaps`, OLS `subjects_vs_soft`, Logit `saturation_vs_sixth`. Output con coefficienti, p-values, R^2 (o pseudo- R^2) e interpretazione testuale auto-generata.

5.4 Distribuzioni

Cinque distribuzioni (carichi docenti, classi per giorno, materia $\times$ slot heatmap, occupancy slot, KS-test, chi-quadro) + goodness-of-fit.

5.5 Run telemetry

Tabella `run_telemetry` (alembic `a848420325b3`). I moduli solver pushano sample via `utils.telemetry.collect` (phase). Il run detail page (`/runs/[id]`) mostra il grafico objective vs tempo (multi-linea per phase) + statistiche per stage + export JSON.

6. API REST

Tutti gli endpoint sono sotto `/api/`. Backend espone ~118 route. Lista non esaustiva delle più rilevanti.

6.1 Pattern CRUD

Ogni risorsa CRUD ha la stessa forma:

Verbo	Path	Descrizione
GET	<code>/api/<entity></code>	lista (q + sort)
GET	<code>/api/<entity>/<id></code>	dettaglio
POST	<code>/api/<entity></code>	create
PUT	<code>/api/<entity>/<id></code>	replace
DELETE	<code>/api/<entity>/<id></code>	delete

6.2 Esempi curl

```
# stato dataset
curl http://127.0.0.1:8000/api/dataset/state

# import small con tutti i pool
curl -X POST -H "Content-Type: application/json" \
  -d '{"profile":"small","use_optimized":true,
      "import_curricula":true,"import_classrooms":true,
      "import_students":true}' \
  http://127.0.0.1:8000/api/dataset/import-profile

# vincolo logico HARD su un docente
curl -X POST -H "Content-Type: application/json" \
  -d '{"expression":"mai aula:LabFisica@mar","kind":"hard"}' \
  http://127.0.0.1:8000/api/teachers/15/logical-unavailabilities

# bulk dry-run su 3 classi
curl -X POST -H "Content-Type: application/json" \
  -d '{"entity_ids":[1,2,3],"action":"add_logical",
      "payload":{"expression":"lun8 AND lun9","is_hard":true,
                  "soft_penalty":100},
      "on_conflict":"dry_run"}' \
  http://127.0.0.1:8000/api/bulk/classes/dry-run

# coverage settimana
curl 'http://127.0.0.1:8000/api/coverage/week?week_start=2026-04-27'

# event lessons (monitor)
curl http://127.0.0.1:8000/api/monitor/event/1/lessons
```


6.3 Riferimento gruppi di endpoint

- **Anagrafiche:** /api/teachers, /classes, /classrooms, /subjects, /curricula, /students, /groups.
- **Cattedre:** /api/assignments + teachers-for-subject + loads.
- **Vincoli logici:** /api/{teachers,classes,classrooms}/{id}/logical-unavailabilities, /api/curricula/{cid}/logical-constraints.
- **Bulk ops:** /api/bulk/{entity}/dry-run|apply.
- **Schedule:** /api/schedule/by-class|by-teacher|by-room|by-slot, move-lesson, move-preview, lesson/{lid}/classroom.
- **Coverage:** /api/coverage/week|cell, /api/absences, /api/substitutions.
- **Optimization:** /api/optimize/assignment|phase-b|lms|sa|ts|ils|full|classroom-assignment.
- **Monitor:** /api/monitor/events|summary|constraints|conflicts, event/{aid}/lessons|lesson/{id}.
- **Import:** /api/import/{entity}, /api/import/{entity}/template.

7. Estendere il sistema

7.1 Aggiungere una nuova tabella + CRUD

1. Modello in `webui/backend/models.py`.
2. Pydantic in `schemas.py` (`XxxIn`, `XxxOut`).
3. Router in `routers/xxx.py`. Copia un router esistente come template.
4. Includi il router in `main.py`: `app.include_router(xxx.router)`.
5. Adapter DSL in `utils/list_query.py`.
6. Frontend: nuova route `webui/frontend/src/routes/xxx/+page.svelte`, riusando *SortableQueryableList*.
7. Voce in `+layout.svelte` per la nav.

7.2 Migrazione idempotente

Aggiorna `db.py::_apply_lightweight_migrations`. Pattern:

```
if insp.has_table("my_table") \
    and not has_column("my_table", "new_field"):
    conn.execute(text(
        "ALTER TABLE my_table ADD COLUMN new_field VARCHAR(64)"
    ))
```

7.3 Aggiungere un nuovo tipo di vincolo

Per-cellula. Copia il pattern di `TeacherUnavailability`: `unique (entity_id, day, hour)`, columns `state/penalty/reason`. Aggiorna `models.py`, `schemas.py`, router, `optimization.py::_availability_constraints`, `routers/monitor.py::_build_constraints` (per il tab Vincoli), `routers/bulk.py::ENTITY_UNAV_MODEL` se vuoi supportare bulk.

Logico. Aggiungi `entity_type` a `LogicalUnavailability`; aggiorna `routers/logical.py::ENTITY_MODEL` e `ENTITY_TYPE`; modifica `routers/monitor.py::_build_constraints`; aggiungi il branch nel solver.

7.4 Aggiungere un nuovo step di ottimizzazione

```
def my_new_step(...):
    params = dict(...)
    run_id = create_run("my_step", "Nome leggibile", profile, params)

    def target(rid: int):
        # carica dati, lancia solver, persiste
        update_run(rid, progress=0.5)
        ...
        update_run(rid, solution_id=sid, obj_value=v, metrics=m)
        update_run(rid, progress=1.0)

    start_thread(run_id, target)
    return run_id
```

Esponi via `routers/optimize.py` con un endpoint POST. Frontend in `/optimize` per il bottone di lancio + log streaming via *RunLogPanel* (SSE).

7.5 Testing

Non c'è una test suite formale. Per smoke-testare end-to-end:

- Backend boot pulito: `uvicorn backend.main:app`.
- Live curl: vedere §7.2.
- Frontend `vite build` deve passare clean.
- Migrazione idempotente: rilancia su DB esistente per confermare zero data loss.

A. Repository GitHub

URL: <https://github.com/gborghi/timetable>.

Repository privato. La landing page contiene il README di alto livello con quickstart, tour della UI, sintassi vincoli, architettura, repository layout.

B. Glossario discorsivo dei metodi

Questo capitolo spiega *cos'è* ogni metodo che π Tantum usa internamente, con un linguaggio che si possa leggere “dal divano”. Per ogni metodo trovi quattro cose in sequenza: l’idea, come funziona, quando usarlo, un esempio legato al timetabling. Niente formule a meno che non aggiungano chiarezza.

B.1 CP-SAT (Constraint Programming over SAT)

Cos'è e perché esiste. CP-SAT è il motore con cui π Tantum trova le soluzioni. È un *constraint solver*: gli dai un mucchio di variabili (per esempio: “in quale slot va la lezione di mate della 1A?”), un mucchio di vincoli (“due lezioni della stessa classe non possono stare nello stesso slot”; “il prof Borghi non insegna sabato”), e gli chiedi di trovare un assegnamento di valori che li rispetti tutti, possibilmente ottimizzando un criterio.

A differenza dei solver *lineari* (LP/MIP) che lavorano su disuguaglianze numeriche, CP-SAT lavora su variabili booleane e intere, e sa esprimere vincoli “logici” complessi (disgiunzioni, implicazioni, conteggi) in modo nativo. Questo lo rende particolarmente adatto al timetabling, dove la maggior parte dei vincoli sono di natura combinatoria, non geometrica.

Come funziona. CP-SAT alterna due fasi:

- **Propagazione:** data una scelta parziale (es. “ho deciso che mate-3A va in lunedì 8^a”), il solver *deduce* automaticamente quello che ne consegue (“allora ita-3A non può andare in lunedì 8^a”; “allora il prof Borghi è occupato in quello slot”, ecc.). Questo restringe lo spazio delle scelte rimanenti senza dover provare tutte le combinazioni.
- **Ricerca:** quando la propagazione esaurisce le conseguenze automatiche, il solver fa un *salto d'azzardo* (es. “provo a mettere ita-3B in martedì 9”) e si rimette a propagare. Se a un certo punto trova una contraddizione, fa *backtracking*: torna indietro, marca quella scelta come “no”, e prova un’alternativa. Impara dai fallimenti aggiungendo *clausole imparate* che evitano di ripetere lo stesso errore.

Quando usarlo. CP-SAT è il default di π Tantum per le Phase A e B. Per scuole di dimensioni normali (fino a ~80 classi) basta da solo. Per istanze più grandi serve combinarlo con la decomposizione spettrale o con metaeuristiche.

Esempio. Tipica iterazione: dopo aver propagato per qualche secondo, il solver ha già piazzato il 70% delle lezioni del triennio (perché sono molto vincolate dai lab e dalle compresenze) ed esplora le combinazioni rimanenti per le classi del biennio (più flessibili). Trova una contraddizione (un docente avrebbe due classi nello stesso slot), *backtracking*, ne prova un’altra. Tipicamente in 1-3 minuti chiude su una soluzione feasible.

B.2 Decomposizione spettrale

Cos'è e perché esiste. Quando la scuola è grande (>120 classi), il problema diventa troppo grosso perché CP-SAT lo digerisca in tempi ragionevoli. La decomposizione spettrale risolve

questo dividendo la scuola in piccoli “cluster” di classi che condividono pochi docenti. Ogni cluster diventa un sotto-problema piccolo, risolvibile in parallelo.

L’idea è: i conflitti veri (due docenti che competono per lo stesso slot) avvengono fra classi che condividono lo stesso docente. Se due classi non hanno docenti in comune, si possono pianificare totalmente in parallelo.

Come funziona. Si costruisce il grafo delle classi: nodi = classi, peso dell’arco fra due classi = numero di docenti condivisi. Si calcolano gli autovettori del *Laplaciano* di quel grafo (una matrice che misura, per ogni nodo, quanto si “differenzia” dai vicini). Gli autovettori associati ai più piccoli autovalori diversi da zero indicano *direzioni di separazione* naturali: progettando i nodi su quelle direzioni, si vedono chiaramente i cluster.

Un colpo di k-means su questa proiezione restituisce la partizione finale.

Quando usarlo. Default ON in π Tantum quando la scuola ha ≥ 120 classi. Per scuole più piccole non porta vantaggi e aggiunge overhead.

Esempio. Per una scuola di 250 classi, una decomposizione in 4 cluster produce sotto-problemi di 60-70 classi ciascuno, risolvibili in parallelo. I docenti-ponte (quelli che insegnano in classi di più cluster) sono il “vincolo di ricucitura” che la fase finale risolve come problema piccolo a se’ stante.

B.3 LNS (Large Neighborhood Search)

Cos’è e perché esiste. Quando hai già una soluzione (anche sub-ottima), spesso è più rapido *aggiustarla* che ricominciare. LNS prende la soluzione attuale, ne “distrugge” una porzione (es. tutte le lezioni di un certo giorno) e la ricostruisce con CP-SAT cercando un miglioramento. Iterando, esplora tante varianti vicine alla soluzione corrente.

Come funziona. A ogni iterazione: scegli un “vicinato” (es. day = martedì), libera tutte le variabili in quel vicinato, lancia CP-SAT con un piccolo budget di tempo per re-piazzarle ottimizzando lo SOFT score. Se il risultato è migliore, lo accetti.

Quando usarlo. Default ON nella pipeline integrata, dopo Phase B. Adatto al “polishing” di una soluzione già feasible.

Esempio. Dopo Phase B la soluzione ha 22 buchi nei docenti. LNS cicla: prova a re-fare lunedì (scende a 20 buchi), martedì (scende a 19), mercoledì (resta 19), ..., dopo qualche minuto si stabilizza intorno a 14-15.

B.4 ALNS (Adaptive LNS)

Cos’è e perché esiste. LNS classico ha un problema: usa sempre lo stesso tipo di vicinato (es. “un giorno”). Su alcune istanze è perfetto, su altre è sterile. ALNS lo evolve: invece di una sola scelta di vicinato, ne ha sei (un giorno intero, un cluster di classi, le ore peggiori della settimana, una giornata di un docente, una giornata di un’aula, una settimana di una sola classe) e tre modi di ricostruire (CP-SAT mini-window, greedy SOFT, BFS riempimento). A ogni iterazione sceglie quale combinazione usare con probabilità proporzionale a un *punteggio* che cresce per le combinazioni storicamente vincenti e decresce per le altre.

Come funziona. È un *roulette wheel selector* sopra punteggi esponenzialmente decadenti. Ogni operatore parte con punteggio neutro; ogni volta che produce un miglioramento, il punteggio sale; ogni volta che fallisce, scende. Aggiunge anche un’accettazione *simulated annealing*-like: occasionalmente accetta peggioramenti per sfuggire ai minimi locali.

Quando usarlo. Sostituisce o segue il LNS classico nella pipeline. Default ON.

Esempio. Su una scuola con cluster ben separati, l’ALNS impara dopo poche iterazioni che “cluster di classi” porta i miglioramenti più grossi e “giornata di un’aula” quasi mai funziona. Lo dice nei log finali con i punteggi.

B.5 SA (Simulated Annealing)

Cos'è e perché esiste. Il nome viene dalla metallurgia: come un metallo riscaldato e poi raffreddato lentamente forma una struttura cristallina più stabile, l'algoritmo accetta inizialmente anche peggioramenti per esplorare lo spazio delle soluzioni, e con il tempo (la “temperatura” che scende) diventa più selettivo, accettando solo miglioramenti.

Come funziona. A ogni iterazione genera una mossa random (es. scambia due lezioni). Se migliora lo SOFT, accetta; se peggiora di Δ , accetta con probabilità $e^{-\Delta/T}$ dove T è la temperatura corrente. La temperatura inizia alta (es. $T_0 = 10$) e si moltiplica per $\alpha < 1$ (es. $\alpha = 0.995$) a ogni iterazione, quindi decresce esponenzialmente.

Quando usarlo. È perfetto quando si sospetta di essere in un minimo locale. Default ON dopo LNS/ALNS.

Esempio. Dopo LNS ti sei stabilizzato a SOFT=420. Lanci SA con $T_0=10$, $\alpha=0.995$, budget 60s. Nei primi secondi, T è alto e accetta peggioramenti fino a 460-470; poi T scende e SA si concentra su miglioramenti, finendo per esempio a 405.

B.6 TS (Tabu Search)

Cos'è e perché esiste. Risolve un problema della ricerca locale: i loop. Se l'unica mossa migliorante è “ $A \rightarrow B$ ” e poi l'unica migliorante da B è “ $B \rightarrow A$ ”, sei intrappolato. TS evita questo tenendo una *lista tabu* delle ultime mosse fatte: per un certo numero di iterazioni, quelle mosse sono vietate. L'algoritmo è così forzato a esplorare zone diverse.

Come funziona. A ogni iterazione, valuta tutte le mosse possibili *tranne* quelle in tabu. Sceglie la migliore, la fa, aggiunge l'inversa alla lista tabu (con un “tempo di scadenza”). La lista è di dimensione fissa (parametro `tabu_size`, default 80).

Quando usarlo. Default ON dopo SA. Particolarmente efficace su istanze con plateau (zone dove molti vicini hanno lo stesso valore).

Esempio. Senza TS, scambiare due lezioni L_1 e L_2 produce SOFT=400; ri-scambiarle riproduce SOFT=405. SA potrebbe oscillare. TS dopo lo scambio mette “inverso di $L_1 \leftrightarrow L_2$ ” in tabu, quindi al passo successivo proverà $L_3 \leftrightarrow L_4$, esplorando una direzione nuova.

B.7 ILS (Iterated Local Search)

Cos'è e perché esiste. Combina ricerca locale con perturbazioni periodiche. Idea: dopo che la ricerca locale si è stabilizzata su un minimo locale, scuoti la soluzione (perturbazione: tante mosse random) e fai ricerca locale da capo. Iterando, esplori bacini di attrazione diversi.

Come funziona. Loop di “cycles” (default 3): in ognuno fai ricerca locale (TS) per un budget; se sei migliorato, salvi; poi perturba (rimuovi e ri-piazza un cluster random di classi); ricomincia.

Quando usarlo. Default ON come ultima fase prima del rooms assignment. È il *closer* della pipeline metaeuristica.

Esempio. Cycle 1 chiude su SOFT=400; perturba e ricomincia da SOFT=480; cycle 2 chiude su SOFT=395; perturba e ricomincia da SOFT=470; cycle 3 chiude su SOFT=392.

B.8 VNS (Variable Neighborhood Search)

Cos'è e perché esiste. VNS è una rifinitura sistematica. Cicla attraverso vicinati di dimensione crescente: prima prova 1-swap (scambia due lezioni), poi 2-swap (due paia), poi 3-chain (catena di 3 mosse), poi k-opt (catena di 4-6). Ogni volta che trova un miglioramento riparte dal vicinato 1; quando un intero ciclo passa senza miglioramenti, si ferma – significa che ha raggiunto un ottimo locale rispetto a tutti i vicinati.

Come funziona. For ogni vicinato, esplora fino a un budget di iterazioni; al primo miglioramento accetta e torna al vicinato 1. Quando il giro $1 \rightarrow 4$ produce zero miglioramenti, esce.

Quando usarlo. Come rifinitura finale, dopo LNS/ALNS/SA/TS. Default OFF (lo accendi quando vuoi spremere il massimo).

Esempio. Hai SOFT=395 dopo ILS. VNS cicla: 1-swap trova un miglioramento a 392; riparte. 1-swap trova ancora a 389; riparte. 1-swap zero; 2-swap trova a 385; riparte. Dopo qualche minuto chiude a 378.

B.9 Hall’s theorem pre-check

Cos’è e perché esiste. Prima di lanciare un solver lungo, conviene controllare se il problema è *strutturalmente* risolubile. Il teorema di Hall (1935) dice: in un *matching bipartito* (assegnare ognuno di N elementi a uno di M elementi compatibili), una soluzione esiste sse per ogni sottoinsieme S degli N elementi, il loro vicinato (gli M compatibili con almeno uno) ha cardinalità $|N(S)| \geq |S|$.

Nel timetabling l’analogo pesato è: per ogni sottoinsieme di docenti, la loro capacità totale di ore deve coprire la domanda totale di ore delle materie che insegnano. Se una materia richiede 40 ore/settimana ma i docenti qualificati ne hanno solo 30 totali, il modello è già infeasible: non c’è bisogno di lanciare CP-SAT.

Come funziona. Tre controlli in cascata: per ogni materia controlla che esista almeno un docente qualificato; per ogni materia controlla che la capacità totale dei docenti qualificati copra la domanda; campiona N sottoinsiemi random di docenti e verifica che la domanda delle materie *esclusive* a ciascun sottoinsieme sia coperta dalla capacità del sottoinsieme.

Quando usarlo. Sempre prima di Phase A. Default ON nella pipeline integrata; se trova violazioni, interrompe immediatamente con un report.

Esempio. Hai aggiunto in mezzo all’anno una nuova materia “Diritto” al triennio (3 ore/settimana per 9 classi = 27 ore) ma hai un solo docente abilitato a Diritto con massimo 18 ore. Hall te lo dice in 100 millisecondi: “Diritto: domanda 27h > capacità docenti 18h”. Risparmi 10 minuti di solver lungo.

B.10 Lagrangian Relaxation con sub-gradient

Cos’è e perché esiste. Quando un problema ha “vincoli scomodi” (vincoli che spezzano una struttura altrimenti separabile), una tecnica classica è *rilassarli*: li rimuovi dal modello e li reintroduci come penalità moltiplicate per coefficienti chiamati *moltiplicatori di Lagrange* (λ). Iterando, i λ si aggiustano per spingere la soluzione del modello rilassato a rispettare comunque i vincoli originari. È un modo di trasformare un problema “a blocchi accoppiati” in una sequenza di sotto-problemi indipendenti.

Nel timetabling questo è utile dopo la decomposizione spettrale: i cluster sarebbero indipendenti se non fosse per i *docenti-ponte* (docenti che insegnano in più cluster). Lagrangian relaxation rilassa la non-sovrapposizione dei docenti-ponte, e itera per riconciliare le scelte dei cluster.

Come funziona. Il *subgradient method* aggiorna λ a ogni iterazione: $\lambda_{k+1} = \max(0, \lambda_k + \alpha_k g_k)$, dove g_k è la violazione del vincolo rilassato e $\alpha_k = \alpha_0 / (1 + k)$ è una sequenza di “passi” che diminuisce a passi piccoli, garantendo convergenza al limite (in teoria) ma in pratica producendo soluzioni utili in poche iterazioni.

Quando usarlo. Scuole con cluster ben separati ma con docenti-ponte fastidiosi. Default OFF (avanzato).

Esempio. Cluster A (40 classi), Cluster B (40 classi), 5 docenti-ponte. Senza Lagrangian, A e B vanno risolti insieme: 80 classi. Con Lagrangian, vengono risolti separatamente in ~ 3 iterazioni: ogni cluster diventa 40 classi (molto più veloce), e λ converge a valori che impediscono ai docenti-ponte di sovrapporsi.

B.11 Column Generation (Dantzig-Wolfe)

Cos'è e perché esiste. Quando un problema è enorme, conviene non enumerarlo tutto: ne generi solo le parti che servono, on-demand. Column Generation è questa filosofia applicata alla programmazione lineare: hai un problema “master” che sceglie da un catalogo di colonne (soluzioni candidate); a ogni iterazione un sotto-problema genera nuove colonne “promettenti” (con *reduced cost* negativo) e le aggiunge al catalogo.

Nel timetabling: ogni docente ha un catalogo di possibili “settimane-tipo” (orari completi per quel docente, già feasible localmente). Il master sceglie quale settimana-tipo prendere per ognuno; il sotto-problema (uno per docente) genera nuove settimane-tipo guidate dal duale del master.

Come funziona. Loop: (1) il master sceglie una combinazione corrente di colonne; (2) calcola i prezzi duali sui vincoli di copertura; (3) ogni sotto-problema produce la nuova colonna più attraente data quei prezzi; (4) se ce n'è una con reduced cost negativo, aggiungila e ripeti; altrimenti hai trovato l'ottimo del rilassamento LP.

Quando usarlo. Solo per scuole molto grandi (>200 classi). Default OFF.

Esempio. 250 classi, 180 docenti. Enumerare tutte le settimane-tipo possibili sarebbe astronomicamente troppo; Column Generation parte con 3 settimane-tipo per docente (540 colonne), ne aggiunge ~50 per iterazione, e converge in 5-10 iterazioni a un orario ottimale rilassato.

B.12 Monte Carlo Sensitivity

Cos'è e perché esiste. Quando hai una soluzione, ti chiedi: *è già vicina al meglio possibile, o c'è ancora margine?* La sensitivity analysis Monte Carlo risponde generando N varianti random della soluzione (ognuna ottenuta applicando 30 mosse atomiche random feasible) e misurando il loro SOFT score. Se la varianza è piccola, sei vicino a un minimo locale; se è grande, hai ancora margine.

Come funziona. Genera N (default 100) walk random di 30 mosse ciascuno, accettando solo quelle che restano HARD-feasible. Per ognuno calcola SOFT. Riporta media, deviazione standard, percentili, coefficiente di variazione $CV = \sigma/\mu$.

Quando usarlo. Dopo aver lanciato la pipeline, per decidere se vale la pena spendere altro tempo in metaeuristiche. $CV < 0.05$ = stai già sull'ottimo locale; $CV > 0.15$ = c'è potenziale di miglioramento.

Esempio. La tua soluzione corrente ha SOFT=400. Monte Carlo restituisce media=402, std=8, $CV \approx 0.02$. Conclusione: la pipeline ha già trovato un buon minimo locale; nuovi giri non porteranno miglioramenti significativi.

B.13 Modularità, betweenness, densità (analisi del bipartito)

Cos'è e perché esiste. Tre metriche dalla teoria dei grafi che ti dicono quanto la struttura della tua scuola sia “decomponibile”. Sono interessanti perché predicono quanto certe tecniche (decomposizione spettrale, parallelismo) funzioneranno bene.

Cosa significano in pratica.

- **Modularità (Newman-Girvan):** misura quanto una partizione *già calcolata* è “naturale”. Valore in $[-1, 1]$. > 0.4 = cluster molto netti, la decomposizione spettrale è perfetta. < 0.2 = la scuola è troppo intrecciata, la decomposizione produrrà cluster artificiosi.
- **Betweenness centrality:** per ogni nodo, conta in quanti percorsi minimi del grafo passa. I nodi ad alta betweenness sono i “ponti”: se hanno SOFT alto, fanno collassare la qualità globale.

- **Densità:** $\frac{2|E|}{|V|(|V|-1)}$, frazione di archi presenti rispetto al massimo. Bassa densità (< 0.1) = scuola sparsa, decomposizione ottima. Alta densità (> 0.5) = quasi cricca, decomposizione inutile.

Quando usarli. Diagnostici da consultare prima di investire in metaeuristiche pesanti: ti dicono che strategia ha senso.

Esempio. Liceo medio: modularità 0.43, betweenness top = “Lingua straniera” (5 prof condivisi su tutti gli indirizzi), densità 0.18. Diagnosi: la decomposizione spettrale produrrà 4 cluster netti (Liceo Classico, Liceo Scientifico, Liceo Linguistico, ITIS); i prof di lingua sono i “ponti” da gestire con cura (Lagrangian relaxation aiuta).

B.14 Distribuzioni e istogrammi

Cosa misurano. 5 viste statistiche dell’orario prodotto: distribuzione del carico orario dei docenti, distribuzione del carico delle classi per giorno, heatmap materia $\times$ slot, occupazione per slot, test di goodness-of-fit (KS contro uniforme, chi-quadro per giornate).

Cosa significano in pratica.

- *Carico docenti:* se la distribuzione è schiacciata su un numero (es. tutti 18) il sistema sta rispettando i tetti di cattedra. Se ha una coda lunga (un prof con 22, un altro con 12), c’è sbilanciamento.
- *Heatmap materia $\times$ slot:* zone scure dove cadono troppe ore di una stessa materia in slot adiacenti.
- *KS-test contro uniforme:* $p > 0.05$ = i carichi sono compatibili con una distribuzione uniforme (giusto); $p < 0.05$ = c’è una distribuzione non casuale (alcuni prof troppo carichi).

Quando usarli. Per dare un giudizio qualitativo sull’orario prodotto, oltre al solo SOFT score numerico.

B.15 Correlazioni e regressioni

Cosa misurano. Tre regressioni statistiche sull’orario corrente:

1. OLS: ore di carico docente $\rightarrow$ numero di buchi.
2. OLS: numero di materie diverse del docente $\rightarrow$ contribuzione SOFT.
3. Logit: saturazione classe $\rightarrow$ probabilità di sesta ora.

Per ognuna ottieni coefficienti, p-value, R^2 .

Cosa significano in pratica. “I docenti con carico > 20 ore hanno in media 2 buchi in più ($p < 0.001$, $R^2 = 0.6$)” = c’è un effetto sistematico documentabile. “La saturazione classe non predice la 6^a ora ($p = 0.4$)” = il sistema sta distribuendo bene le ore e non concentra le classi sature in 6^a.

Quando usarle. Per riportare al collegio docenti dati *quantitativi* sulla qualità dell’orario, anziché solo affermazioni qualitative.

B.16 DSL generico

Cos’è e perché esiste. I sub-DSL precedenti (query lista, logical DNF, objective Phase A, introspettivo) risolvevano ognuno un caso, ma erano grammatiche separate. Ogni fix andava

replicato 4 volte. Il *DSL generico* unifica: una grammatica sola, parser unico, quattro back-end di compilazione.

Come si rapporta al modello dati. Le sorgenti iterabili (*lessons*, *teachers*, *classes*, *classrooms*, *students*, *groups*, ...) sono viste pre-calcolate del modello SQLAlchemy: a ogni valutazione, il sistema “materializza” queste viste in dict Python che il parser interroga.

Come si traduce. Un `forall l in lessons where ...: predicate` si traduce, per il backend EVAL, in un ciclo Python su tutti gli elementi che soddisfano il filtro, applicando il predicato. Per il backend LOGIC (vincoli logical DNF) si traduce in clausole disgiuntive di letterali. Per il backend OBJECTIVE si traduce in una somma pesata aggiunta alla funzione obiettivo della Phase A. Stessa sintassi, traduzioni diverse.

Quando usarlo. Per qualunque vincolo che non rientri nelle preferenze standard. Vedi capitolo dedicato per la gallery di 30+ esempi.

B.17 Stati 5-colori delle matrici

Cos'è. Le matrici di disponibilità (docente, classe, aula) hanno 5 stati ortogonali per ogni cella, con colori distintivi per leggerli a colpo d'occhio:

- **HARD** (rosso): cella vietata in modo assoluto.
- **ENFORCED** (rosso scuro): cella che *deve* esserci occupata.
- **PREFERRED** (verde): preferenza positiva (bonus).
- **DISLIKED/SOFT** (giallo): preferenza negativa (penalità).
- **ALLOWED** (bianco): neutro.

Scorciatoie tastiera. Selezionata una cella o un range, premere H/E/P/D/A la imposta in HARD/ENFORCED/ PREFERRED/DISLIKED/ALLOWED. N per “neutro” (alias ALLOWED). Click + drag per selezionare un range; doppio click sull'header di colonna o riga per applicare a tutta la colonna/riga.

B.18 Tag aule e tag studenti

Cos'è. Etichette libere che il coordinatore attacca a un'aula o a uno studente. Sono *many-to-many*: un'aula può avere tanti tag, un tag può essere su tante aule.

Perché esistono. I tag rendono i vincoli più robusti al cambiamento dell'anagrafica. Invece di scrivere “solo queste tre aule precise” (lista che andrebbe aggiornata a ogni modifica), scrivi “aule taggate matematica”: se domani aggiungi una nuova aula taggata allo stesso modo, il vincolo la include automaticamente.

Casi d'uso tipici.

- Tag aule: *matematica*, *proiettore*, *lab_lingue*, *piano_terra*, *accessibile_disabili*.
- Tag studenti: *BES* (Bisogni Educativi Speciali), *DSA*, *debito_matematica_4*, *pcto_Acme*, *studente_atleta*.

I tag NON sostituiscono i gruppi articolati (che sono unità operative di scheduling); sono metadato passivo per filtrare e raggruppare nell'interfaccia.

B.19 Run telemetry e visualizzazione live

Cos'è. Ogni esecuzione del solver produce una *serie temporale* di sample: ad ogni iterazione il sistema annota timestamp, fase corrente, valore obiettivo, mosse accettate/rifiutate, vincoli HARD violati. Tutto salvato in DB e accessibile dal detail page del run.

Cosa vedi. Un grafico interattivo (Apache ECharts) con l'evoluzione del SOFT score nel tempo, una linea diversa per ogni stage (LNS in azzurro, SA in verde, TS in giallo, ...), markers verticali sui cambi di stage. Statistiche aggregate per stage in tabella (n. iterazioni, durata, best obj). Esportazione in JSON per analisi esterne.

Live mode. Per i run in corso, la pagina poll-a il backend ogni 2s e aggiorna il grafico man mano che arrivano nuovi sample. La pagina pausa il polling automaticamente quando la tab del browser è in background, per non sprecare banda.

B.20 Lockati: lezioni che non si toccano

Cos'è. Il flag `locked` sulla tabella `Lesson` indica che quella lezione *non deve essere spostata* dal solver. Utile per fissare a mano una manciata di lezioni critiche e poi lanciare l'ottimizzazione su tutto il resto.

Come si usa. Dal Monitor (e dalla griglia Schedule con click-destro), il bottone “Blocca” marca la lezione come locked. Il bottone “Sblocca” fa l'inverso. Le lezioni locked sopravvivono ai run successivi: vengono caricate come *vincoli aggiuntivi HARD* dal solver, che dovrà incastrare il resto attorno a esse.

Il filtro “Lockati” nella tab del Monitor mostra solo gli eventi che hanno almeno una lezione lockata, utile per visualizzare velocemente cosa hai già fissato e cosa è ancora libero.

C. Reference

- Pisinger, "Where are the hard knapsack problems?", 2005.
- Kohl, Karisch, Patriksson, "Very Large-Scale Neighborhood Search Techniques", 2002.
- Burke, Kingston, Pepper, "A standard data format for timetabling instances", 2008 (S1877050919318800).
- Ahuja, Magnanti, Orlin, "Network Flows", 1993.